

Echomix mix network design motivation

EJ Infeld

1. Mix networks - introduction

Mix networks, or *mixnets*, are an anonymous communications network paradigm distinguished by striking a balance between practicality and protection from strong adversaries. A mix network consists of network devices, typically referred to as mix nodes, that relay messages between clients in such a way that an adversary is unable to determine which clients are communicating with each other. This is done by reordering or mixing multiple indistinguishable messages.

There must be some trade-off in latency - packets must not be forwarded from a node immediately, in order to be *mixed* with other packets, and some trade-off in bandwidth - packets must be padded to a common size. They also have to be re-encrypted at each hop. [1] The notion that both latency and bandwidth overhead must be non-zero was formalized in [2]. Additional trade-offs occur with adding common anonymity strategies, such as decoy traffic. It is thanks to these combined strategies and trade-offs that carefully constructed mix networks can offer protection against global adversaries. Because of these trade-offs mix networks are not able to simulate browsing the Internet comfortably as a VPN might, and are typically considered for use with some services, such as messaging.

Many mix network designs and protocols have been published, starting in 1979 [1], and we observe a rise in commercial efforts to build mixnets [3]–[5]. Much of the focus of these efforts is in incentivisation mechanisms for mix node operators, and financial privacy. The largest of these, Nym [3], is based on the Loopix model. [6] Due to a subtle oversight in design, Loopix is vulnerable to a GPA as described in [subsubsection 2.4.3](#). It also has a single point of failure in a user's Service Provider, making it easy for an active adversary to compromise a target user.

2. The Echomix design

The Echomix system contains three server node types: gateways, mix nodes and services. None of these has a persistent relationship with clients. Service nodes are positioned on the far side of the network, and, in contrast to leading mix network designs, every interaction with them is a round trip - an *echo*. When a client connects to a random gateway, the Sphinx [7] packets sent by the client are relayed through the three layers of mix nodes, and then to a service. A reply is sent, which can confirm delivery or contain send query results. A service can send this reply without knowing the location of the client thanks to a Sphinx Single-Use Reply Block (SURB).

In the context of a messaging application, a message travelling from Alice to Bob requires two *echos* to the far side of the network, one from Alice to write a message

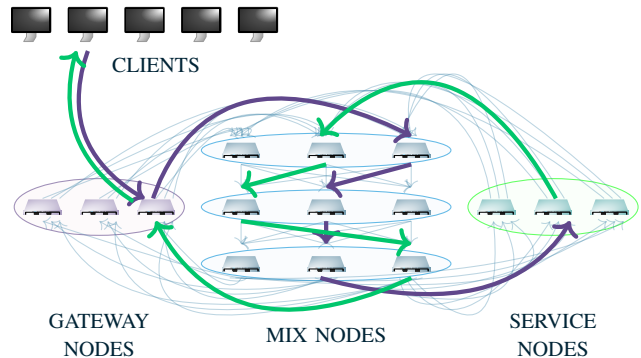


Figure 1: A client's interaction with the service node is a round-trip, with a packet's forward route marked in purple, and service's confirmation in green. The intermediate node layers are pictured from top to bottom.

to a service, and a second initiated by Bob to retrieve it. This symmetry and a careful construction of protocols, allows us to out-perform other anonymous communication systems in resisting traffic analysis, malicious providers, and confirmation attacks.

The clients generate a stream of *echos* to uniformly random, or pseudorandom - in the case of some applications, including message streams - services, independently of whether a client requests a service or not. Most of these packets will be decoy traffic. The stream is a single Poisson process, with delays at each hop sampled from an exponential distribution, as in [8] and [6]. The advantages of this strategy are described in section 2.4.1. Unlike in Loopix [6], the amount of sent and received traffic is fully independent of whether the packets are application traffic or decoys.

From the point of view of network architecture, Echomix, and its implementation - Katzenpost [9], is an Internet overlay mix network atop TCP/IP or QUIC/IP. The subsequent layers are governed by the following protocols.

- 1) PQ Noise [10] transport protocol.
- 2) Sphinx [7] or PQ-Sphinx routing protocol.
- 3) Application layer, such as Pigeonhole messaging.

Katzenpost is the first software implementation of the PQ Noise. As a transport protocol it enforces the network topology, e.g. mix nodes in layer 1 are only allowed to downstream-connect to the mix nodes in layer 2. All PQ Noise messages are padded to a uniform size.

Since all application protocols are built based on packet round-trips to services positioned on the far side of the network, the Sphinx ability for the service to send a reply without knowing the location of the client is used throughout the design. This is done with Single Use Reply Blocks (SURBs). We describe our post quantum updates

to the Sphinx format in ??.

The Katzenpost PKI is an adjacent protocol at the root of authority within the mix network. Directory authorities publish PKI documents every epoch, distributing network connection information and public key materials to the nodes. This system is similar to Tor's. [11] Providing all nodes and clients with a uniform view of the network allows us to resist epistemic attacks. [12] An adversary who compromises a majority of directory authorities compromises the entire mix network. We therefore employ a decentralised PKI design, relying on multiple independent directory authority operators. The consensus producing the PKI document uses the post quantum hybrid signature scheme of Ed25519 [13] combined with Sphincs+ [14].

2.1. Gateway nodes

In contrast to [6], we believe that the nodes on the edge of the network should have as little information on the client behavior as possible, as they are the ones that can identify the client. Gateways have no persistent relationship with the client, and no knowledge of whether any of the client's packets are decoys or not.

If the client traffic crossing the node is low, the gateways additionally generate decoy traffic. We propose a new heuristic, the *Coupon Collector's bound* in order to ensure that all links of the network are active, resulting in a reliable mixing of packets.

Coupon Collector's Bound. *If packets released from a node are directed to one of the nodes in the next layer uniformly at random, the expected number of messages that node has to release before at least one message has been directed to each node in the subsequent layer behaves like $\Theta(n \log(n))$.*

Proof: This is a direct consequence of the Coupon Collector's Problem. $\square$

Let:

- $\mu = 1/\lambda$ be the mean time that a packet lingers in a node due to memoryless mixing,
- n be the maximum number of nodes in a layer,
- g be the number of gateways.

We aim to achieve a high probability that during each period μ , there is at least one packet crossing a given link in the network. The Coupon Collector's Problem tells us how many packets we need to cover links from the gateways to the first layer of mixes. In order for all links between two layers of size n to be reliably active, we should additionally multiply the desired output of a gateway by n/g . Therefore in order for all links to be active with high probability, and therefore for best mixing of the packets in the network, each gateway should be outputting at least an average of $\Theta(n^2 \log(n)/g)$ packets in the time μ . In practice, this bound is significantly lower than the number of packets travelling through the nodes.

2.2. Decoy traffic and application traffic

An *echo* decoy packet is a Sphinx round-trip through the mix network to a service node sampled uniformly at random. A service request to a server sampled in a way indistinguishable from uniformly random, followed by a

service response sent back with the SURB, is unobservably coupled to echo decoy traffic.

Traffic coupling. *Let sequences A and B be two probabilistic processes on states $S = \{s_1, s_2, \dots, s_n\}$. If both A and B are indistinguishable from uniformly random, then any process C which selects either A or B to sample the next state according to any algorithm A is also indistinguishable from uniformly random.*

Proof: Let $\mathcal{H}$ be any state history in C , and $s_j \in S$ be any state. Then for any values of $p = \mathbb{P}_A(A|\mathcal{H})$ and $p' = \mathbb{P}_B(B|\mathcal{H})$, the probability of the next element being from sequence A and sequence B respectively, the probability of the next sampled state being s_j is

$$p \times \mathbb{P}_A(s_j|\mathcal{H}) + p' \times \mathbb{P}_B(s_j|\mathcal{H}) = (p + p') \times \frac{1}{n} = \frac{1}{n}.$$

A does not have to be independent of history $\mathcal{H}$. $\square$

If and only if we maintain the correct coupling, the user's application traffic is fully unobservable within decoy traffic, and causes no anomalies, avoiding the shortcomings of leading systems described in subsection 2.4.3. For receiving messages, maintaining unobservability requires care, since not only are the queries correlated with the related send requests, but we may have to contend with the dangers of multiple queries to the same server, which may be statistically significant and therefore visible to a passive observer. The introduction of *couriers* (??) allows us to efficiently maintain unobservably coupled traffic. It also prevents *SURB floods* - an attack in which a malicious service might collect multiple linkable SURBs and send them in a burst in order to identify a client.

2.3. Service nodes

Service nodes are positioned behind the mix network and handle functionality requested by the client. This could be storing messages, publishing information outside of the mixnet, interfacing with a blockchain node etc. They also process and execute the SURBs of decoy packets. Any application should be constructed so that the following conditions are met:

- 1) Separate service requests of a client are unlinkable. Repeating the same request may be linkable.
- 2) Services are treated uniformly by the client, with no persistent relationship.
- 3) The traffic from a client to the service is correctly coupled with the decoy traffic.

The last condition means that either the service is chosen in a way that is independent from traffic history and indistinguishable from uniformly random for each query to a service, or the packet will replace a decoy packet that was meant to go to the specific service. Since the latter introduces additional latency, we achieve the former in several ways in sections ?? and ??.

2.4. Iterating on prior research

So far, we described original additions to the field of mix network design. For the remainder of this section, we will discuss how we implement and improve upon established concepts, and motivate the new design by pointing

out vulnerabilities of preceding systems. Beginning with ??, we list remaining original contributions which allow us to extend our security guarantees to persistent messaging, and resisting quantum adversaries.

Katzenpost grew out of the Panoramix project [15], and was previously an evolution of Loopix [6], which combined ideas of sampling an exponential distribution (memoryless mixing) to delay at each hop [8], monitoring system health with heartbeat traffic [16], organizing nodes in a layered topology [17], assigning persistent Service Providers for each user at the edge of the network, and wrapping data in Sphinx packets [7], a packet format designed specifically for mix networks.

2.4.1. Memoryless mixing. Echomix adopts node delays sampled from an exponential distribution, as introduced in [8] and used in Loopix [6]. This distribution has the advantage of being *memoryless* - at each point in time, each message sitting in a mix will have the same probability distribution of the remaining delay, independently of how long it has already been waiting. This means that for an external observer the probability distribution of which message will be sent next is uniform at all times. For a constant parameter $\lambda > 0$ with the mean $1/\lambda$, the delays approximate the probability distribution function:

$$f(x_{\geq 0}, \lambda) = \lambda e^{-\lambda x}.$$

However, [8] claims incorrectly that the behavior of a resulting mix node is Poisson distributed. This is then reproduced in [6], culminating in nodes of these type being called *Poisson mixes*. The behavior of a sum of multiple Poisson processes is in fact only Poisson if the set of processes being summed doesn't change over time. We propose to call this type of mixing *memoryless mixing* instead, while noting that the overall behavior of the node itself is not memoryless and a node should not be called a *memoryless mix*.

2.4.2. Heartbeat packets. All nodes in the mixnet generate *heartbeat packets* [16], which are sent through the system before returning to their origin. This allows each node to take stock of the functioning of the network - if any segment of the route is disrupted, packets crossing through that segment will not come back in the expected time window. This allows the system to detect an $n - 1$ attack [18], or any other attack that involves a disruption of the network. However, heartbeat packets not originating from the clients are only effective in detecting faulty or malicious behavior in some mix nodes. Malicious nodes on the edge of the network can distinguish between mix heartbeats and user requests and decide to forward any heartbeat packet that is scheduled to return to a mix node, while dropping user traffic meant to exit the network.

Additionally, no node or client in the network should act on these measurements by itself. Such a process could result in different clients having different views of the network and opening the system to *epistemic* [12] and *compulsion* [19] attacks. Instead, nodes in Echomix process the collected measurements [20] and rate the links' health, and then upload the ratings to directory authorities. The directory authorities establish a consensus and distribute the updated structure of the network to all parties in the next PKI document, once per consensus epoch.

Finally, both [16] and [6] describe rating nodes in the mixnet. It is more helpful to rate links on the route, rather than nodes. Not only does it provide finer data, and allow for detection of $n - 1$ attacks directly, but in practice many networking problems happen specifically between two locations. A separate argument in favor of indexing over links can be found in [21] for batch mixes.

2.4.3. Vulnerability to traffic analysis in Loopix. This Loopix design vulnerability has not been addressed elsewhere. A legitimate message traveling from the last layer of mixes is sure to go to the receiver's designated provider, as opposed to decoy traffic from the last layer of mixes, which is uniformly distributed among providers. Application traffic at the last hop is therefore independently overlaid on decoy traffic.

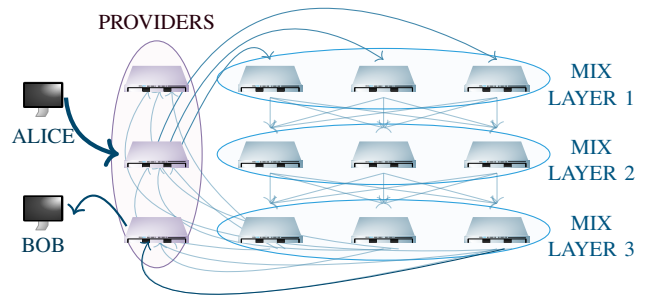


Figure 2: In Loopix, as Alice communicates with Bob, the increase in traffic to Bob's Provider is observable.

This means that application traffic at that hop is observable to a passive network adversary, as long as it is statistically significant, and especially if there is any regularity to it. In this situation, the other traffic at this hop is not an effective cover, it's noise, and in many real-life situations a signal processing analysis could easily do away with it. This is dangerous even in the short term, since the low latency between the sender and the receiver's Provider opens this system to correlation attacks and SURB floods, and any advantage that comes from asynchronicity is lost.

A naive solution if one wanted to retain the rest of the Loopix design might be to wait until a decoy packet is going to the right provider, and send the message instead. This would significantly reduce the available bandwidth by effectively dividing it by the number of Providers. In sections ?? and ??, we carefully design a correct coupling of decoy and application traffic which does not reduce the bandwidth.

2.4.4. Providers. Loopix features persistent *Service Providers* that the clients connect to directly. The Provider is a significant point of failure, with access to a trove of a user's information, including the message receiving pattern. It was suggested in [6] that returning heartbeat traffic may double as decoy traffic for received messages, but this is incorrect. These streams are independently overlaid, and can be similarly decoupled as above when a client is online. When the receiver is not online, loop traffic is not present at all. In Echomix we only include gateways on the perimeter of the network, with service

providers positioned behind it, and do not include persistent providers or gateways for a user at all. Interactions of the same user are unlinkable by the service provider.

2.5. The challenges of persistent multi-client messaging

Persistent messaging between multiple clients comes with an additional set of statistical and active attacks, and a need for second party anonymity. The design described so far is appropriate for a wide array of applications, including one-to-all anonymous publishing, interfacing with blockchain nodes, and other sender-only use cases. It is a significant improvement on the security properties of previously published mixnet designs. In order to further extend it to persistent messaging, we introduce the BACAP and Pigeonhole suite of protocols to allow for messaging while maintaining our security guarantees. These are described in sections ?? and ??.

References

- [1] David L. Chaum. Untraceable electronic mail, return addresses, and digital pseudonyms. *Commun. ACM*, 24(2):84–90, feb 1981.
- [2] Debajyoti Das, Sebastian Meiser, Esfandiar Mohammadi, and Aniket Kate. Anonymity trilemma: Strong anonymity, low bandwidth overhead, low latency—choose two. *Cryptology ePrint Archive*, Paper 2017/954, 2017. <https://eprint.iacr.org/2017/954>.
- [3] Claudia Diaz, Harry Halpin, and Aggelos Kiayias. The Nym Network <https://nymtech.net/nym-whitepaper.pdf>.
- [4] <https://0kn.io/>.
- [5] <https://hoprnet.org/>.
- [6] Ania M. Piotrowska, Jamie Hayes, Tariq Elahi, Sebastian Meiser, and George Danezis. The loopix anonymity system. In *26th USENIX Security Symposium (USENIX Security 17)*, pages 1199–1216, Vancouver, BC, August 2017. USENIX Association.
- [7] George Danezis and Ian Goldberg. Sphinx: A compact and provably secure mix format. In *2009 30th IEEE Symposium on Security and Privacy*, pages 269–282, 2009.
- [8] Dogan Kesdogan, Jan Egner, and Roland Büschkes. Stop- and go-mixes providing probabilistic anonymity in an open system. volume 1525, pages 83–98, 04 1998.
- [9] <https://github.com/katzenpost>.
- [10] Yawning Angel, Benjamin Dowling, Andreas Hülsing, Peter Schwabe, and Florian Weber. Post quantum noise. *IACR Cryptology ePrint Archive*, 2022, 2022.
- [11] <https://spec.torproject.org/dir-spec/index.html>.
- [12] George Danezis and Paul Syverson. Bridging and fingerprinting: Epistemic attacks on route selection. volume 5134, pages 151–166, 07 2008.
- [13] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. In Bart Preneel and Tsuyoshi Takagi, editors, *Cryptographic Hardware and Embedded Systems – CHES 2011*, pages 124–142, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [14] Daniel J. Bernstein, Daira Hopwood, Andreas Hülsing, Tanja Lange, Ruben Niederhagen, Louiza Papachristodoulou, Michael Schneider, Peter Schwabe, and Zooko Wilcox-O’Hearn. Sphincs: Practical stateless hash-based signatures. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015*, pages 368–397, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.
- [15] Privacy and accountability in networks via optimized randomized mix-nets, grant agreement id: 653497. <https://cordis.europa.eu/project/id/653497>.
- [16] George Danezis and Len Sassaman. Heartbeat traffic to counter (n-1) attacks: Red-green-black mixes. In *Proceedings of the 2003 ACM Workshop on Privacy in the Electronic Society, WPES ’03*, page 89–93, New York, NY, USA, 2003. Association for Computing Machinery.
- [17] Claudia Diaz, Steven J. Murdoch, and Carmela Troncoso. Impact of network topology on anonymity and overhead in low-latency anonymity networks. In Mikhail J. Atallah and Nicholas J. Hopper, editors, *Privacy Enhancing Technologies*, pages 184–201, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [18] Andrei Serjantov, Roger Dingledine, and Paul Syverson. From a trickle to a flood: Active attacks on several mix types. volume 2578, 02 2003.
- [19] George Danezis and Jolyon Clulow. Compulsion resistant anonymous communications. pages 11–25, 06 2005.
- [20] David Stainton, Yawning Angel, and Masala, 2022. <https://github.com/katzenpost/katzenpost/blob/main/core/pki/document.go>.
- [21] Hemi Leibowitz, Ania Piotrowska, George Danezis, and Amir Herzberg. No right to remain silent: Isolating malicious mixes, 09 2018.